Guia de estudo - Prova 1 (COMP0506)

Prova: Aula 15, 14/10 (segundo o plano de ensino da T04).
Livro: RUSSELL, Stuart J.; NORVIG, Peter. *Artificial intelligence: A modern approach*, global edition. Harlow: Pearson, 2022 (4ª edição).
Assuntos: agentes inteligentes; busca em largura e profundidade; busca A*; interação de agentes; construção do conhecimento com lógica.

---

0. Onde estudar cada assunto

Assunto	Slides do professor (Drive)	Livro (4ª ed.)	Nossa atividade
Agentes inteligentes	Aula 02 e Aula 03	Cap. 2 (2.1 a 2.4)	Parte 1 de `atividade-ia-agentes-barcelona.md` (PEAS e ambiente)
Formulação de problemas	Aula 04 - Parte 1	Cap. 3 (3.1 a 3.3)	Grafo da Parte 3
Largura e profundidade	Aula 04 - Partes 1 e 2	3.4.1, 3.4.3 e 3.4.4	Atividade "fila e pilha" da Aula 04
A*	Aula 04 - Partes 1 e 2	3.5.1 a 3.5.4	Partes 3 e 4 de `atividade-ia-agentes-barcelona.md`
Interação de agentes	Atividade AgentesBusca, notebooks `RoboSocial` e `AgentesSociais`	2.3.2 (multiagente) e 18.1	Parte 2 de `atividade-ia-agentes-barcelona.md`
Lógica e base de conhecimento	`agentes_logicos_aula8` e Atividade Construção de Base de Conhecimento	Cap. 7 (7.1 a 7.5) e 8.1, 9.1, 9.2	`atividade-ia-logica-agente.md`

Plano sugerido para os dois dias:

1. **Dia 1:** seção 5 (lógica) do zero, com calma, e os exercícios de lógica. É o assunto em que você disse ter mais dificuldade.
2. **Dia 2, manhã:** seções 3 e 4 (largura, profundidade e A*). Refaça à mão o exemplo da Romênia.
3. **Dia 2, tarde:** seções 1 e 2 (agentes e interação), os exercícios restantes e o checklist final.

---

1. Agentes inteligentes

Conceitos básicos

- **Agente:** qualquer coisa que **percebe** o ambiente por **sensores** e **age** sobre ele por **atuadores**.
- Humano: olhos e ouvidos; mãos e pernas.
- Robô: câmeras; motores.

- Software: teclado e arquivos; tela e envio de pacotes.
- **Percepção:** o que o agente recebe num instante.
- **Sequência de percepções:** tudo o que ele já percebeu (a memória).
- **Função do agente:** mapeia uma sequência de percepções para uma ação.
- **Exemplo clássico, o aspirador de pó:** dois quadrados, A e B. Regra: "se o quadrado atual está sujo, aspira; senão, vai para o outro".

Racionalidade

Um **agente racional** escolhe a ação que **maximiza o desempenho esperado**, dado o que percebeu até agora e o que sabe. Isso depende de quatro fatores:

1. a medida de desempenho (o critério de sucesso);
2. o conhecimento prévio do ambiente;
3. as ações possíveis;
4. a sequência de percepções até o momento.

Racional não é o mesmo que onisciente. O exemplo dos slides: a pessoa atravessa a rua olhando para os lados e é atingida por algo que cai de um avião. Ela agiu racionalmente, só não tinha como saber. Racional também não quer dizer perfeito.

- O desempenho é medido pelos **estados do ambiente**, não pela opinião do agente sobre si mesmo.
- **Coleta de informação:** olhar para os lados antes de atravessar faz parte de ser racional.
- **Autonomia:** o agente que aprende com as percepções vai além do conhecimento inicial que o projetista deu.

PEAS: especificar o ambiente de tarefa

Letra	Significado	Exemplo: táxi automatizado (Aula 03)
Performance	Medida de desempenho	Chegar ao destino, segurança, menos tempo e consumo, conforto, lucro
Environment	Ambiente	Ruas e rodovias, tráfego, pedestres, clima
Actuators	Atuadores	Volante, acelerador, freio, tela, voz
Sensors	Sensores	Câmeras, sonar, GPS, velocímetro, sensores do motor

O Wumpus World (Aula 03 e Aula 8) também tem PEAS e cai muito em prova:

- **Desempenho:** +1000 pelo ouro, -1000 por morrer, -1 por ação, -10 por usar a flecha.
- **Sensores:** fedor, brisa, brilho, choque e grito.

Propriedades do ambiente (decore os pares)

Dimensão	Opção 1	Opção 2	Exemplo
Observabilidade	Totalmente observável	Parcialmente observável	O táxi não sabe o que os outros motoristas pensam: parcial
Agentes	Agente único	Multiagente (competitivo ou cooperativo)	Xadrez é competitivo; evitar colisão é cooperativo
Determinismo	Determinístico	Estocástico	Trânsito é estocástico
Episódios	Episódico	Sequencial	Classificar peças é episódico; dirigir é sequencial
Dinâmica	Estático	Dinâmico (ou semidinâmico)	Palavras cruzadas é estático; dirigir é dinâmico
Valores	Discreto	Contínuo	Xadrez é discreto; posição e velocidade são contínuas
Regras	Conhecido	Desconhecido	Xadrez é conhecido; um videogame novo é desconhecido

Semidinâmico: o ambiente não muda, mas o desempenho cai com o tempo (por exemplo, xadrez com relógio).

Na nossa atividade, o cruzamento foi classificado como parcialmente observável, multiagente cooperativo, estocástico, dinâmico e contínuo.

Tipos de agente (livro, seção 2.4)

1. **Reativo simples:** "se condição, então ação"; olha só a percepção atual. Exemplo: o aspirador.
2. **Reativo baseado em modelo:** guarda um **estado interno**, uma memória de como o mundo está. É o "agente com estado" da Aula 8 (perceber, atualizar o estado, escolher a ação).
3. **Baseado em objetivos:** planeja para alcançar uma meta. É aqui que entra a **busca**.
4. **Baseado em utilidade:** escolhe pelo "quão bom" é o resultado, não só "chegou ou não".
5. **Com aprendizagem:** melhora com a experiência.

Representações do estado (Aula 02)

- **Atômica:** o estado é uma "caixa preta", só um nome (por exemplo, "estou em Arad"). É o que a busca usa.
- **Fatorada:** o estado é um vetor de atributos (posição, combustível, GPS). Lida com valores desconhecidos.
- **Estruturada:** objetos e as relações entre eles ("caminhão bloqueado por uma vaca"). É o caminho da lógica de primeira ordem.

---

2. Interação de agentes

Multiagente

- **Competitivo:** um ganha quando o outro perde (xadrez).
- **Cooperativo:** todos ganham com o mesmo resultado (evitar colisões).
- **Na nossa cena:** pedestre, motorista e semáforo cooperam pela segurança.

O que o professor cobrou na atividade (Atividade AgentesBusca)

1. **Força social:** distância mínima de conforto entre pedestres. - Base: proxêmica de Hall (zona pessoal de 0,45 a 1,2 m) e o modelo de força social de Helbing e Molnár (repulsão que cresce quando alguém se aproxima). - Nós usamos **0,8 a 1,2 m** como conforto e **0,4 a 0,5 m** como risco de contato.
2. **Prioridade de passagem:** quem cede num corredor estreito. - Quem já está no meio da travessia mantém o impulso. - O pedestre isolado desvia do grupo. - O táxi cede ao pedestre.
3. **Impacto do histórico:** a **sequência de percepções** (memória) deixa o agente prever que o vizinho mudou de direção e desviar antes. Sem memória, cada quadro seria um mundo novo.

Notebook `RoboSocial` (robô socialmente consciente)

- A cada frame, o robô monta um **grafo dinâmico** com a posição dele, o objetivo e os pedestres dentro do **raio de visão**.
- **g(n):** distância real + **multa de +50** se a rota passa a menos de 15 unidades de um pedestre.
- **h(n):** distância euclidiana até o objetivo.
- Com a força social ligada, passar perto de gente fica "caro", e o A* faz desvios mais amplos e educados.
- Se o A* não encontra caminho, o robô usa uma rota de emergência em linha reta.
- Parâmetros para entender:
- aumentar a multa (+200) gera voltas enormes;
- diminuir (+5) faz o robô quase ignorar as pessoas;
- aumentar a bolha (de 15 para 30) faz o robô se afastar antes.

Notebook `AgentesSociais` (sociedade de agentes)

- Grade 50x50 com prédios (obstáculos) e três "tribos": trabalhadores vão ao escritório, turistas ao parque (e são mais lentos), estudantes à cafeteria.
- Navegação por **Weighted A***: **f(n) = g(n) + w·h(n)**, com w = 1,2 e heurística **Manhattan** (|x1 - x2| + |y1 - y2|).
- **Interação social:** quando dois agentes ficam vizinhos, há uma probabilidade de "conversarem", e eles pausam o movimento.

Weighted A* em uma frase: com w > 1, a busca confia mais na heurística. Fica mais rápida, mas **perde a garantia de ótimo**: o custo pode chegar a até w vezes o ótimo (livro, 3.5.4).

Ambiente dinâmico e replanejamento

Num ambiente dinâmico, o grafo muda enquanto o agente anda. Ele precisa **replanejar** quando o mapa muda. Na nossa Parte 4, o semáforo é o coordenador: cada troca de fase muda o grafo, e o pedestre e o motorista rodam o A* de novo.

---

3. Formulação de problemas e busca sem informação

Os 5 componentes de um problema (Aula 04)

Componente	Exemplo: Romênia
Estado inicial	Em(Arad)
Ações	Ir(Sibiu), Ir(Timisoara), Ir(Zerind)
Modelo de transição	RESULTADO(Em(Arad), Ir(Zerind)) = Em(Zerind)
Teste de objetivo	Em(Bucareste)?
Custo de caminho	Soma das distâncias em km

Outros exemplos dos slides:

- **Aspirador:** 8 estados (2 posições × sujeira em A × sujeira em B); ações Esquerda, Direita e Aspirar; cada passo custa 1.
- **Quebra-cabeça de 8 peças:** 181.440 estados alcançáveis; cada movimento custa 1.

Vocabulário da busca

- **Árvore de busca:** a raiz é o estado inicial, os ramos são as ações e os nós são os estados.
- **Expandir** um nó é gerar os filhos dele.
- **Borda (fronteira):** nós gerados que ainda não foram expandidos.
- **Conjunto explorado:** nós já expandidos. A **busca em grafo** usa esse conjunto para não repetir estados nem cair em laços (Arad - Sibiu - Arad...).
- **b:** fator de ramificação (quantos filhos por nó).
- **d:** profundidade da solução mais rasa.
- **m:** profundidade máxima da árvore.

Busca em largura (BFS)

- Expande **nível por nível**.
- A borda é uma **fila FIFO**: o primeiro a entrar é o primeiro a sair.
- Encontra a solução **mais rasa**, com menos passos.

Busca em profundidade (DFS)

- Desce até o fundo de um ramo e depois volta.
- A borda é uma **pilha LIFO**: o último a entrar é o primeiro a sair.
- Usa pouquíssima memória.

Exemplo resolvido (faça à mão)

Grafo: I → A, I → B; A → C, A → D; B → G; D → G. Objetivo: **G**. Os filhos entram em ordem alfabética, e o objetivo é testado quando o nó **sai** da borda.

Largura (fila; sai pela esquerda, entra pela direita):

Passo	Sai da fila	Fila depois
0	-	[I]
1	I	[A, B]
2	A	[B, C, D]
3	B	[C, D, G]
4	C	[D, G]
5	D	[G] (G já está na fila, não repete)
6	**G, objetivo**	-

Caminho: I → B → G, com **2 passos**, o mais raso.

Profundidade (pilha; o topo fica à esquerda):

Passo	Sai da pilha	Pilha depois
0	-	[I]
1	I	[A, B]
2	A	[C, D, B]
3	C	[D, B] (C não tem filhos)
4	D	[G, B]
5	**G, objetivo**	-

Caminho: I → A → D → G, com **3 passos**. Achou rápido, mas **não** o mais raso. Por isso a DFS não é ótima.

Comparação (cai em prova)

Algoritmo	Borda	Completa?	Ótima?	Tempo	Espaço
Largura	Fila FIFO	Sim (b finito)	Sim, se todo passo tem o mesmo custo	O(b^d)	O(b^d), o problema é a memória
Profundidade	Pilha LIFO	Só em espaço finito (sem laços)	Não	O(b^m)	**O(b·m)**
Profundidade limitada (limite l)	Pilha	Não, se l < d	Não	O(b^l)	O(b·l) no livro (o slide escreve O(b^l))

---

4. Busca informada e A*

Heurística

- **h(n)** é uma **estimativa** do custo do nó n até o objetivo.
- No objetivo, h = 0.
- Exemplo clássico: a **distância em linha reta (DLR)** até Bucareste.

Busca gulosa

- Expande o nó com **menor h(n)**, ou seja, f(n) = h(n).
- É rápida, mas **não é ótima nem completa** (pode entrar em laço).

A*

- **f(n) = g(n) + h(n)**:
- **g(n)** é o custo **real** do início até n;
- **h(n)** é o custo **estimado** de n até o objetivo.
- Sempre expande o nó da borda com **menor f**.
- Exemplo do slide: g = 300 e h = 100 dão f = 400.
- Só termina quando o objetivo **sai** da borda. Se ele apenas foi gerado, ainda não termina.

Quando o A* é ótimo

- **Admissível:** h(n) **nunca superestima** o custo real. A distância em linha reta é admissível, porque nenhum caminho é menor que a reta.
- **Consistente:** para todo vizinho n' de n, **h(n) ≤ c(n, n') + h(n')** (a desigualdade triangular).
- Toda consistente é admissível, mas o contrário não vale.
- **Busca em árvore:** basta ser admissível. **Busca em grafo:** precisa ser consistente.
- O A* é **otimamente eficiente**: com a mesma heurística, nenhum algoritmo ótimo expande menos nós.
- **Limitação: memória**, porque o A* guarda todos os nós gerados. Variantes que economizam memória: IDA* e outras.
- Quanto melhor a heurística, mais os **contornos** de f se alinham com o caminho ótimo e menos nós são expandidos.

Exemplo resolvido: Romênia, de Arad a Bucareste

Valores de h (distância em linha reta até Bucareste):

Cidade	h
Arad	366
Sibiu	253
Timisoara	329
Zerind	374
Fagaras	176
Rimnicu Vilcea	193
Oradea	380
Pitesti	100
Craiova	160
Bucareste	0

Passo	Expande (menor f)	Gera (g + h = f)
1	Arad (0 + 366 = 366)	Sibiu 140+253=**393**; Timisoara 118+329=447; Zerind 75+374=449
2	Sibiu (393)	Fagaras 239+176=415; Rimnicu 220+193=**413**; Oradea 291+380=671
3	Rimnicu Vilcea (413)	Pitesti 317+100=417; Craiova 366+160=526
4	Fagaras (415)	Bucareste 450+0=450 (entra na borda, mas **não** termina)
5	Pitesti (417)	Bucareste 418+0=**418** (melhor que 450)
6	**Bucareste (418), fim**	-

Rota ótima: Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucareste, com custo **418**.

Compare com a busca gulosa, que segue só o h: Arad → Sibiu → Fagaras → Bucareste, com custo **450**, que não é o ótimo.

Por isso o slide diz que "Pitesti é expandido antes de Bucareste": 417 < 450.

Como fizemos o A* nas atividades

- **Pedestre:**
- grade de 0,5 m (a largura de um corpo), com passo ortogonal de 0,5 e diagonal de cerca de 0,71;
- g = distância + penalidade social (+1, +5 ou +50 conforme a distância) + táxi (+50);
- h = distância euclidiana.
- O A* contorna o grupo porque atravessá-lo deixa g alto.
- **Motorista:**
- trechos de 5 m, com um nó que guarda posição e instante;
- esperar 3 s custa 18 e avançar com gente na faixa custa 115, então o A* escolhe esperar.
- **Semáforo:** não roda A*; ele muda o grafo dos outros dois.

---

5. Lógica do zero

5.1 O que é uma proposição

Uma **proposição** é uma frase que só pode ser **verdadeira (V)** ou **falsa (F)**. Damos uma letra a ela:

- P: "há um pedestre muito próximo"
- C: "o corredor está estreito"

"Que horas são?" não é proposição, porque não é nem V nem F.

Cuidado com a notação: nos slides, às vezes aparece **"V"** no lugar do símbolo ∨ ("ou"). Por exemplo, "P[1,2] V P[2,1]" quer dizer "P[1,2] **ou** P[2,1]". Não confunda com V de verdadeiro.

5.2 Os operadores (conectivos)

Símbolo	Nome	Lê-se	É verdadeiro quando...
¬P	Negação	"não P"	P é falso (inverte)
P ∧ Q	Conjunção	"P **e** Q"	**os dois** são V
P ∨ Q	Disjunção	"P **ou** Q"	**pelo menos um** é V
P ⇒ Q	Implicação	"**se** P, **então** Q"	só é **F** quando P é V e Q é F
P ⇔ Q	Bicondicional	"P **se e somente se** Q"	P e Q têm o **mesmo** valor

Tabela de cada operador. Decore, porque é a base de tudo:

P	Q	¬P	P ∧ Q	P ∨ Q	P ⇒ Q	P ⇔ Q
V	V	F	V	V	**V**	V
V	F	F	F	V	**F**	F
F	V	V	F	V	**V**	F
F	F	V	F	F	**V**	V

Como lembrar a implicação. Pense numa promessa: "**Se** chover, **então** levo guarda-chuva."

- Choveu e levei (V, V): cumpri a promessa, então **V**.
- Choveu e **não** levei (V, F): **quebrei** a promessa, então **F**. É o único caso falso.
- Não choveu, levei ou não levei (F, V ou F, F): a promessa não foi testada, então **V**.

Ordem de precedência (do mais forte para o mais fraco): **¬, depois ∧, depois ∨, depois ⇒, depois ⇔**.

- ¬P ∧ Q significa (¬P) ∧ Q.
- P ∧ C ⇒ Y significa (P ∧ C) ⇒ Y.
- Na dúvida, use parênteses.

5.3 Como montar uma tabela-verdade, passo a passo

Exemplo: **(P ∧ ¬Q) ⇒ R**

Passo 1. Conte as variáveis e as linhas. São 3 variáveis (P, Q, R), então há 2^3 = **8 linhas**. Com n variáveis, são 2^n linhas.

Passo 2. Preencha as variáveis com o padrão de metades.

- A primeira coluna tem metade V e metade F: 4 V e depois 4 F.
- A segunda alterna de 2 em 2.
- A última alterna de 1 em 1.

Assim você nunca esquece nem repete uma combinação.

Passo 3. Crie uma coluna para cada pedaço da fórmula, de dentro para fora: primeiro ¬Q, depois P ∧ ¬Q e por fim (P ∧ ¬Q) ⇒ R.

P	Q	R	¬Q	P ∧ ¬Q	(P ∧ ¬Q) ⇒ R
V	V	V	F	F	V
V	V	F	F	F	V
V	F	V	V	V	V
V	F	F	V	V	**F**
F	V	V	F	F	V
F	V	F	F	F	V
F	F	V	V	F	V
F	F	F	V	F	V

Passo 4. Leia o resultado. A fórmula só é falsa quando P = V, Q = F e R = F, porque é o único caso em que a parte do "se" é V e a do "então" é F.

Nomes úteis:

- **Tautologia:** V em todas as linhas (exemplo: P ∨ ¬P).
- **Contradição:** F em todas as linhas (exemplo: P ∧ ¬P).
- **Satisfazível:** V em pelo menos uma linha.

5.4 Equivalências que caem em prova

Duas fórmulas são **equivalentes (≡)** quando têm a mesma coluna final na tabela-verdade.

Equivalência	Nome
¬(¬P) ≡ P	Dupla negação
P ⇒ Q ≡ ¬P ∨ Q	Eliminação da implicação
P ⇒ Q ≡ ¬Q ⇒ ¬P	Contrapositiva
¬(P ∧ Q) ≡ ¬P ∨ ¬Q	De Morgan
¬(P ∨ Q) ≡ ¬P ∧ ¬Q	De Morgan
P ⇔ Q ≡ (P ⇒ Q) ∧ (Q ⇒ P)	Eliminação do bicondicional

Armadilha: P ⇒ Q **não** é equivalente a Q ⇒ P (a recíproca). "Se chove, a rua molha" não significa "se a rua está molhada, choveu", porque alguém pode ter lavado a rua.

5.5 Base de conhecimento (KB) e o agente lógico

- **KB (Knowledge Base):** um conjunto de sentenças em linguagem formal que o agente "sabe".
- **Abordagem declarativa:** você **diz (TELL)** à KB o que ela precisa saber e **pergunta (ASK)** o que fazer; a resposta tem que decorrer do que foi dito.
- **Motor de inferência:** os algoritmos, que não dependem do domínio, tiram conclusões da KB.
- Ciclo do agente (pseudocódigo da Aula 8): TELL(percepção), depois ASK(qual ação?), depois TELL(ação escolhida), depois t + 1.

5.6 Consequência lógica (entailment) e modelos

- Um **modelo** é um "mundo possível", ou seja, uma linha da tabela-verdade.

- **KB ⊨ α** (lê-se "a KB acarreta α") quando α é **verdadeira em todos os modelos em que a KB é verdadeira**. Em símbolos: M(KB) ⊆ M(α).
- **Verificação de modelos:** monta a tabela, olha **só as linhas em que a KB é V** e confere se α também é V em todas elas.
- Se α for V em todas, então KB ⊨ α.
- Se houver **uma única** linha com a KB V e α F, então KB ⊭ α.
- **Custo:** 2^n linhas, que cresce exponencialmente (7 símbolos já dão 128 linhas).

Exemplo do Wumpus (Aula 8). Pij quer dizer "há poço em [i,j]" e Bij quer dizer "há brisa em [i,j]".

- R1: ¬P11
- R2: ¬B11 (sem brisa em [1,1])
- R3: B21 (brisa em [2,1])
- R4: B11 ⇔ (P12 ∨ P21)
- R5: B21 ⇔ (P11 ∨ P22 ∨ P31)

Raciocínio:

- Como **B11 é falso**, R4 obriga P12 ∨ P21 a ser falso. Logo, **¬P12** e ¬P21: **[1,2] é segura** (KB ⊨ α1).
- Como **B21 é verdadeiro**, R5 obriga P11 ∨ P22 ∨ P31 a ser verdadeiro. P11 é falso, então **P22 ∨ P31**: o poço está em [2,2] **ou** em [3,1].
- Existe um modelo com poço em [2,2], então **não dá para provar** que [2,2] é segura (KB ⊭ α2).

5.7 Regras de inferência e encadeamento

Modus Ponens (a regra que usamos na atividade): de **P ⇒ Q** e **P**, concluímos **Q**.

- Inválido (falácia): de P ⇒ Q e Q concluir P.

Cláusula de Horn: um fato (A) ou uma regra do tipo (A ∧ B ∧ ...) ⇒ C. Com KB só de cláusulas de Horn, o Modus Ponens é completo e a inferência é rápida (linear).

Exemplo dos slides. A KB tem:

- as regras P ⇒ Q, L ∧ M ⇒ P, B ∧ L ⇒ M, A ∧ P ⇒ L e A ∧ B ⇒ L;
- os fatos A e B.

Encadeamento para frente (orientado a dados): dispara toda regra cujas premissas já são conhecidas, até chegar à pergunta.

1. A e B são conhecidos; por **A ∧ B ⇒ L**, conclui **L**.
2. B e L; por **B ∧ L ⇒ M**, conclui **M**.
3. L e M; por **L ∧ M ⇒ P**, conclui **P**.
4. P; por **P ⇒ Q**, conclui **Q**. Pergunta respondida.

Encadeamento para trás (orientado a objetivos): parte da pergunta Q e procura regras que concluam o que falta.

1. Quero Q. A regra P ⇒ Q exige provar **P**.
2. Quero P. A regra L ∧ M ⇒ P exige provar **L** e **M**.
3. Quero L. A regra A ∧ B ⇒ L funciona, porque A e B são fatos. L está provado.
4. Quero M. A regra B ∧ L ⇒ M funciona, porque B é fato e L foi provado. M está provado.
5. Logo P está provado, e Q também.

	Para frente	Para trás
Começa por	Fatos	Pergunta
Estilo	Orientado a dados	Orientado a objetivos
Problema	Pode deduzir coisas irrelevantes	Precisa evitar ciclos (pilha de objetivos)
Exemplo do slide	Reconhecimento automático, rotina	"Onde estão minhas chaves?"

5.8 Lógica de primeira ordem (só o básico da Aula 8)

- A lógica proposicional só tem **fatos**. Ela não consegue dizer "poços causam brisa em **todas** as casas vizinhas" sem escrever uma regra para cada casa.
- A **primeira ordem** tem:
- **objetos** (pessoas, casas);
- **relações** (Irmão(x, y), Vermelho(x));
- **funções** (Pai(x));
- os quantificadores ∀ ("para todo") e ∃ ("existe").
- **Instanciação universal:** de ∀x Rei(x) ∧ Ganancioso(x) ⇒ Malvado(x), tira-se Rei(John) ∧ Ganancioso(John) ⇒ Malvado(John).
- **Unificação:** achar substituições que deixam duas sentenças iguais.
- UNIFY(Knows(John, x), Knows(John, Jane)) = {x/Jane}
- UNIFY(Knows(John, x), Knows(y, Bill)) = {x/Bill, y/John}
- UNIFY(Knows(John, x), Knows(x, Elizabeth)) = **falha**, porque x não pode ser John e Elizabeth ao mesmo tempo.

5.9 Como a lógica liga com o A* (nossa atividade)

A **lógica decide o quê** (ceder, evitar o caminho) e o **A* decide por onde**. A conclusão lógica vira custo:

- B ("evitar o caminho atual") verdadeiro significa penalidade em g(n) no caminho atual;
- ¬A ("não pode avançar") verdadeiro significa que as células ficam bloqueadas.

No nosso exemplo, o atalho custa 4,0 + 5 = **9,0** e o contorno custa 5,42 + 0 = **5,42**, então o A* escolhe o contorno.

---

6. Exercícios (gabarito no fim)

Para treinar no formato da prova (bot com enunciado comum, A* contra largura, tabela-verdade e caminho falho), faça também o simulado (PDF).

1. Faça o PEAS de um **robô de entrega num shopping** e classifique o ambiente nas 7 dimensões.
2. No grafo S → A, S → B; A → C; B → D, B → G; C → G, com filhos em ordem alfabética, mostre a **fila** da BFS e a **pilha** da DFS até achar G. Qual caminho cada uma encontra?
3. Arestas: S-A = 1, S-B = 4, A-B = 2, A-G = 12, B-G = 3. Heurística: h(S) = 5, h(A) = 4, h(B) = 2, h(G) = 0. - (a) Rode o A* mostrando f em cada passo. - (b) A heurística é admissível? É consistente? - (c) Que caminho a busca gulosa acharia?
4. Monte a tabela-verdade de **¬P ∨ Q** e compare com a de **P ⇒ Q**. O que você conclui?
5. Monte a tabela-verdade de **(P ∨ Q) ∧ ¬P**. Em que linha ela é verdadeira?
6. A KB = {P ⇒ Q, P}. Vale KB ⊨ Q? E a KB = {P ⇒ Q, Q}: vale KB ⊨ P? Justifique com um modelo.
7. KB: A, B, A ∧ B ⇒ C, C ⇒ D, D ∧ A ⇒ E. Use encadeamento para frente para provar E.

8. Por que aumentar a multa social de +50 para +200 no notebook `RoboSocial` faz o robô dar voltas maiores? E o que muda com w = 1,2 no Weighted A*?
9. Por que a busca em largura costuma esgotar a memória antes do tempo?
10. Explique a diferença entre agente **racional** e agente **onisciente** com um exemplo.

Gabarito

1. Uma resposta possível:

- **P (desempenho):** entregar no lugar e no tempo certos, sem colisões, respeitando o espaço pessoal, gastando pouca bateria.
- **E (ambiente):** corredores, lojas, pedestres, escadas, portas.
- **A (atuadores):** rodas, compartimento de carga, tela e voz.
- **S (sensores):** câmeras, LIDAR ou sonar, odometria, bateria.
- **Classificação:** parcialmente observável, multiagente cooperativo, estocástico, sequencial, dinâmico, contínuo e conhecido (ou parcialmente conhecido).

2. Largura e profundidade:

- **BFS:** [S], depois [A, B], [B, C], [C, D, G], [D, G] e [G]; G sai da fila. Caminho **S-B-G**, com 2 passos.
- **DFS** (topo à esquerda): [S], depois [A, B], [C, B] e [G, B]; G sai da pilha. Caminho **S-A-C-G**, com 3 passos (não é o mais raso).

3. A*, heurística e gulosa:

- **(a) Passos do A*:** 1. Expande S (f = 5) e gera A (g = 1, f = 5) e B (g = 4, f = 6). 2. Expande A (5): B por A tem g = 3 e f = 5, melhor que o anterior; G tem g = 13 e f = 13. 3. Expande B (5): G por B tem g = 6 e f = 6, melhor que 13. 4. Expande G (6) e termina. Caminho **S-A-B-G**, com custo **6**.
- **(b)** É admissível: o custo real a partir de S é 6, de A é 5 e de B é 3, e todos os h são menores ou iguais. Também é consistente: h(S) = 5 ≤ 1 + 4, h(A) = 4 ≤ 2 + 2 e h(B) = 2 ≤ 3 + 0.
- **(c)** A gulosa vai de S para B (porque h = 2 < 4) e de B para G: caminho S-B-G com custo **7**, que não é o ótimo.

4. As colunas finais são iguais (V, F, V, V). Portanto **P ⇒ Q ≡ ¬P ∨ Q**.

5.

P	**Q**	**P ∨ Q**	**¬P**	**(P ∨ Q) ∧ ¬P**
V	V	V	F	F
V	F	V	F	F
F	V	V	V	**V**
F	F	F	V	F

Só é verdadeira com P = F e Q = V.

6. Os dois casos:

- **Sim, KB ⊨ Q (Modus Ponens):** a única linha em que P e P ⇒ Q são V tem Q = V.
- **Não, KB ⊭ P:** no modelo P = F e Q = V, a KB é verdadeira e P é falso (é a falácia de afirmar o consequente).

7. De A e B, por A ∧ B ⇒ C, conclui C. Por C ⇒ D, conclui D. De D e A, por D ∧ A ⇒ E, conclui **E**.

8. A multa entra em g(n). Com +200, qualquer caminho perto de gente fica tão caro que um desvio longo passa a ter f menor. Com w = 1,2, o h pesa mais: a busca vai mais direto ao objetivo e expande menos nós, mas pode devolver um caminho até 1,2 vezes mais caro que o ótimo.

9. A BFS guarda a borda inteira de cada nível: O(b^d) nós na memória. A memória acaba antes do tempo disponível.

10. O racional maximiza o desempenho **esperado** com o que percebeu; o onisciente saberia o resultado real das ações. Exemplo: atravessar olhando para os lados é racional, mesmo que caia algo do céu, coisa que só um onisciente saberia.

---

7. Material externo (gratuito)

Vídeos em português

- Professor Douglas Maioli: Matemática Discreta (playlist): as aulas 1 a 4 são exatamente a lógica que você precisa (1: tabela-verdade dos conectivos; 2: tautologia e contradição; 3: equivalências; 4: regras de inferência). **Comece por aqui.** O site do professor tem slides, ficha-resumo de lógica e exercícios resolvidos.
- e-Aulas da USP: PRG0010 Inteligência Artificial (Fabio Cozman): curso que segue o mesmo livro.
- Vídeos 5 a 8: solução de problemas por busca.
- Vídeos 13 a 17: lógica.
- Playlist "Introdução à Inteligência Artificial", do canal AlgoritmoZ (40 vídeos curtos, de cerca de 10 min):
- #2: Agentes e Racionalidade: agentes, função do agente e o aspirador de pó, igual à Aula 02.
- #3: Ambiente de Tarefas e PEAS: a tabela PEAS e as propriedades do ambiente. É o vídeo logo depois do #2 na playlist.
- Algoritmos de busca, Parte 2: largura e profundidade: mostra a fronteira como fila e como pilha, passo a passo.

Vídeos em inglês (Berkeley CS188, usa o mesmo livro)

- Vídeos curtos "passo a passo": SBS-1 "DFS and BFS" e SBS-2 "A* Search" resolvem exemplos à mão em poucos minutos. As aulas completas 2 (Uninformed Search) e 3 (Informed Search) estão na mesma página.
- Playlist CS188 Fall 2018: inclui a aula "A* Search and Heuristics".

PEAS e propriedades do ambiente (slides em português, com exemplos e exercícios)

- CIn/UFPE: Agentes Inteligentes: PEAS e as propriedades do ambiente, no mesmo roteiro do capítulo 2 do livro.
- UFABC (Folivetti): Aula 02: tabela PEAS do "motorista de Uber" e exercícios para fazer PEAS de jogar futebol, fazer compras no mercado e dar lances num leilão.
- Slides oficiais do livro, capítulo 2 (inglês): PEAS do táxi e do agente de compras na internet, e a tabela que classifica paciência, gamão, compras na internet e táxi em cada propriedade.

Listas de exercícios

- Aula prática de procura da Universidade Nova de Lisboa: segue a 4ª edição do livro e tem tabelas passo a passo de largura, profundidade, gulosa e A* (`X(g+h=f)`), admissibilidade e consistência, e verdadeiro ou falso. Está em português de Portugal: "procura" quer dizer busca e "sôfrega" quer dizer gulosa.
- Simulic: BFS, DFS e A* num labirinto: simulação interativa em português, com perguntas de reflexão.
- Material da UFPE sobre agentes lógicos: slides em português com Wumpus e encadeamento para frente e para trás.

Livro e código oficiais

- Sumário oficial do AIMA: use para achar as seções citadas acima.
- aima-python: código e notebooks do livro em Python. Os notebooks `search.ipynb` e `logic.ipynb` seguem os capítulos 3 e 7.

Busca (visual, muito bom para entender)

- Red Blob Games: Introduction to A*: animações interativas comparando largura, Dijkstra, gulosa e A*. É o melhor material para "ver" a borda crescendo.
- PathFinding.js: desenhe paredes numa grade e rode BFS, DFS, gulosa e A* com heurística Manhattan ou euclidiana.

Lógica

- Gerador de tabela-verdade de Stanford (CS103): digite a fórmula (por exemplo `(p /\ ~q) -> r`) e confira as suas tabelas. Monte primeiro à mão e use a ferramenta só para corrigir.

Interação de agentes

- O artigo de Helbing e Molnár (1995) sobre o modelo de força social, já citado na nossa atividade.
- Livro, seção 18.1 (propriedades de ambientes multiagente: cooperação e coordenação).

---

8. Checklist da véspera

☐ Sei definir agente, função do agente e agente racional, e diferenciar racional de onisciente.
☐ Sei montar o PEAS de qualquer cenário e classificar o ambiente nas 7 dimensões.
☐ Sei os 5 componentes de um problema de busca.
☐ Sei rodar BFS (fila) e DFS (pilha) à mão e dizer qual é completa e qual é ótima.
☐ Sei calcular f = g + h e rodar o A* da Romênia sem olhar.
☐ Sei explicar admissível e consistente e por que a gulosa não é ótima.
☐ Sei explicar força social, prioridade de passagem, histórico de percepções e replanejamento.
☐ Sei as tabelas de ¬, ∧, ∨, ⇒ e ⇔ de cor.
☐ Sei montar uma tabela-verdade com 3 variáveis (8 linhas) pelo padrão das metades.
☐ Sei o que é KB ⊨ α e verificar com modelos.
☐ Sei aplicar Modus Ponens e encadeamento para frente e para trás.